

SWE 4502: Design Pattern Lab

Afrin Jahan Era

ID: 220042132

Department: CSE

Programme: SWE

DP_Lab_06_Report

Here -

Houses are complex objects with many optional parts. Different house types require different configurations. Construction steps must follow a fixed order. Creating a separate builder for every possible feature combination leads to an exponential number of builder classes.

Design Pattern Used: Builder Pattern

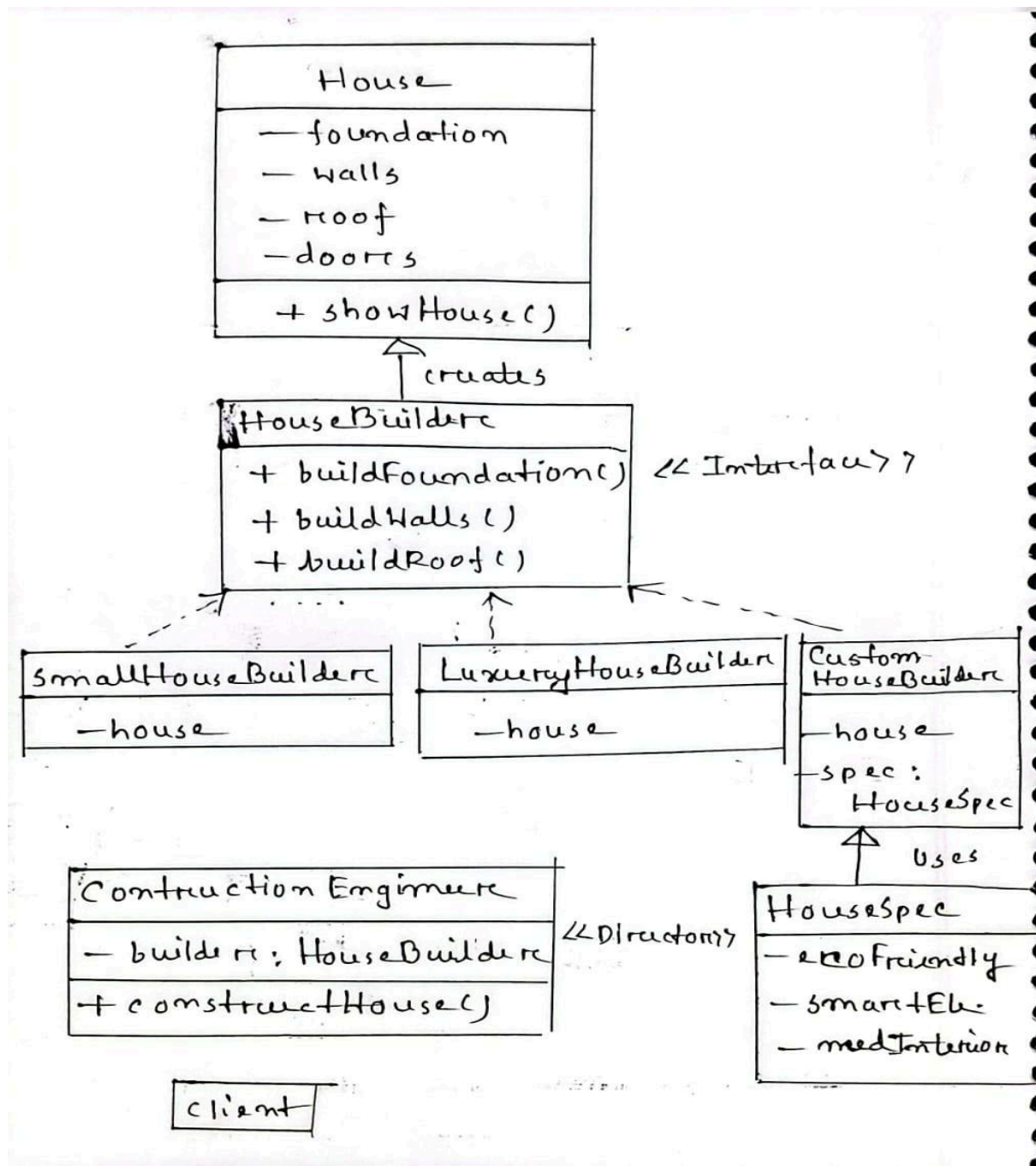
The Builder Pattern separates the construction of a complex object from its representation, allowing the same construction process to create different representations. I decide to design this in this way :-

1. Product: House
2. Builder Interface: HouseBuilder
3. Concrete Builders: SmallHouseBuilder, LuxuryHouseBuilder, CustomHouseBuilder
4. Director: ConstructionEngineer

System Design Overview

The HouseBuilder interface defines all construction steps. Concrete builders implement these steps differently. The ConstructionEngineer (Director) controls the order of construction. The client selects the house type. A HouseBuilderFactory returns the appropriate builder.

Explanation



The hand-drawn diagram illustrates the Builder Pattern applied to the House Construction System. The House class represents the final product containing multiple parts. The HouseBuilder interface defines step-by-step construction methods. Concrete builders implement these steps according to house type.

The ConstructionEngineer acts as the Director and ensures the construction steps are executed in the correct order. To avoid creating many builders for different feature combinations, a CustomHouseBuilder is used along with a HouseSpec configuration object. This allows multiple house variations using a single builder class.

Solution to Exponential Combination Problem

Instead of creating a new builder class for every possible feature combination, the design uses:

- HouseSpec (Configuration Object) – Stores feature flags such as eco-friendly, smart electrical, and interior requirements.
- CustomHouseBuilder – Reads the configuration and conditionally builds house components.

This approach reduces class explosion and keeps the system flexible and maintainable.

Implementation Code (Java)

Product – House

```
class House {
    String foundation;
    String walls;
    String roof;
    String doors;
    String windows;
    String electrical;
    String interior;

    void showHouse() {
        System.out.println("House Details:");
        System.out.println("Foundation : " + foundation);
        System.out.println("Walls      : " + walls);
        System.out.println("Roof      : " + roof);
        System.out.println("Doors     : " + doors);
        System.out.println("Windows   : " + windows);
        System.out.println("Electrical : " + electrical);
        System.out.println("Interior  : " + interior);
    }
}
```

Builder Interface

```
interface HouseBuilder {  
    void buildFoundation();  
    void buildWalls();  
    void buildRoof();  
    void buildDoorsAndWindows();  
    void buildElectrical();  
    void buildInterior();  
    House getHouse();  
}
```

Concrete Builder – SmallHouseBuilder

SmallHouseBuilder and LuxuryHouseBuilder implement the interface to build different house types using different materials and designs.

```
class SmallHouseBuilder implements HouseBuilder {  
  
    private House house = new House();  
  
    public void buildFoundation() {  
        house.foundation = "Simple concrete foundation";  
    }  
    public void buildWalls() {  
        house.walls = "Brick walls";  
    }  
    public void buildRoof() {  
        house.roof = "Asphalt roof";  
    }  
    public void buildDoorsAndWindows() {  
        house.doors = "Wooden doors";  
        house.windows = "Single-glass windows";  
    }  
    public void buildElectrical() {  
        house.electrical = "Basic electrical setup";  
    }  
    public void buildInterior() {  
        house.interior = "Simple interior design";  
    }  
    public House getHouse() {  
        return house;  
    }  
}
```

Concrete Builder – LuxuryHouseBuilder

```
class LuxuryHouseBuilder implements HouseBuilder {  
  
    private House house = new House();  
  
    public void buildFoundation() {  
        house.foundation = "Deep reinforced foundation";  
    }  
    public void buildWalls() {  
        house.walls = "Concrete and marble walls";  
    }  
    public void buildRoof() {  
        house.roof = "Designer tiled roof";  
    }  
    public void buildDoorsAndWindows() {  
        house.doors = "Designer wooden doors";  
        house.windows = "Double-glazed windows";  
    }  
    public void buildElectrical() {  
        house.electrical = "Smart electrical system";  
    }  
    public void buildInterior() {  
        house.interior = "Luxury interior design";  
    }  
    public House getHouse() {  
        return house;  
    }  
}
```

Solving Exponential Combination Problem

Configuration Class – HouseSpec

```
class HouseSpec {  
    boolean ecoFriendly;  
    boolean smartElectrical;  
    boolean needInterior = true;  
}
```

Parametrized Builder – CustomHouseBuilder

Here ONE builder handles MANY combinations

```
class CustomHouseBuilder implements HouseBuilder {

    private House house = new House();
    private HouseSpec spec;

    CustomHouseBuilder(HouseSpec spec) {
        this.spec = spec;
    }

    public void buildFoundation() {
        house.foundation = spec.ecoFriendly ?
            "Eco-friendly foundation" : "Standard foundation";
    }

    public void buildWalls() {
        house.walls = "Standard walls";
    }

    public void buildRoof() {
        house.roof = "Standard roof";
    }

    public void buildDoorsAndWindows() {
        house.doors = "Standard doors";
        house.windows = "Standard windows";
    }

    public void buildElectrical() {
        house.electrical = spec.smartElectrical ?
            "Smart electrical system" : "Basic electrical system";
    }

    public void buildInterior() {
        if (spec.needInterior) {
            house.interior = "Custom interior";
        } else {
            house.interior = "Not included";
        }
    }

    public House getHouse() {
        return house;
    }
}
```

Director – ConstructionEngineer

```
class ConstructionEngineer {  
  
    private HouseBuilder builder;  
  
    ConstructionEngineer(HouseBuilder builder) {  
        this.builder = builder;  
    }  
    public House constructHouse() {  
        builder.buildFoundation();  
        builder.buildWalls();  
        builder.buildRoof();  
        builder.buildDoorsAndWindows();  
        builder.buildElectrical();  
        builder.buildInterior();  
        return builder.getHouse();  
    }  
}
```

Client Code – Main

```
public class Main {  
    public static void main(String[] args) {  
  
        // Client directly selects the builder  
  
        HouseBuilder builder = new SmallHouseBuilder();  
        // OR  
        // HouseBuilder builder = new LuxuryHouseBuilder();  
        // OR  
        // HouseBuilder builder = new CustomHouseBuilder(spec);  
        // Director controls construction  
  
        ConstructionEngineer engineer = new ConstructionEngineer(builder);  
        // Build house step by step  
        House house = engineer.constructHouse();  
        // Show result  
        house.showHouse();  
    }  
}
```